

Sparse Table

1. Overview

A **Sparse Table** is a static data structure that precomputes answers for all intervals whose length is a power of two. Once built, it can answer range queries (like minimum, maximum, GCD) in **$O(1)$** time.

Key characteristics:

- **Static** — The underlying array must not change after the table is built. Updates are not supported.
- **Idempotent-friendly** — It excels at operations where overlapping intervals can be combined safely (e.g., `min`, `max`, `gcd`, `bitwise AND`, `bitwise OR`).
- **$O(n \log n)$ preprocessing** — The table is built in $O(n \log n)$ time and memory.
- **$O(1)$ query** — Any range query reduces to combining two precomputed intervals.

The sparse table is a workhorse for placement and competitive programming problems that ask many static range queries on an immutable array.

2. Intuition

Simple Explanation

Imagine you have an array of size `n`. You want to answer "what is the minimum value from index `L` to `R`?" — and you have to answer thousands of such queries.

If you check the range each time, that's $O(n)$ per query. If you precompute every possible interval, that's $O(n^2)$ memory — too much.

The **sparse table** finds a sweet spot: precompute intervals of lengths 1, 2, 4, 8, 16, ... (powers of two). For a query of any length, you can **cover it with two overlapping precomputed intervals** whose combined length is a power of two, and because `min/max/gcd` are **idempotent** (overlapping doesn't double-count), the answer is just `min(precomputed_A, precomputed_B)`.

Analogy

Think of having a ruler with markings at every power of two: 1 cm, 2 cm, 4 cm, 8 cm, etc. To measure any distance, you take the two largest power-of-two markings that fit inside it, letting them overlap a little. Since you're looking for the coldest (minimum) temperature in that span, overlapping is fine — you just take the minimum of the two measurements.

Step-by-Step Reasoning

1. Let `st[k][i]` store the answer for the range `[i, i + 2k - 1]` — that is, a length- 2^k interval starting at `i`.
2. `st[0][i]` is just `arr[i]` (interval of length 1).
3. For $k \geq 1$, a length- 2^k interval can be split into two length- 2^{k-1} intervals:
 - `st[k][i] = min(st[k-1][i], st[k-1][i + 2(k-1)])`
4. For a query `[L, R]`, let `len = R - L + 1`. Let $k = \text{floor}(\log_2(\text{len}))$.

- `Answer = min(st[k][L], st[k][R - 2^k + 1])`
- These two intervals exactly cover the query with overlap.

Why It Works

- Every interval length can be written as a binary number.
- The largest power of two $\leq$ length, when placed twice (once at the left edge, once at the right edge), covers the whole range with guaranteed overlap.
- For **idempotent operations** (min, max, gcd, bitwise AND/OR), overlapping does not affect correctness. You can safely take the op of the two.
- For **non-idempotent operations** (sum, XOR), overlapping would double-count, so sparse table does not work — use a segment tree or Fenwick tree instead.

3. When to Use It

Use a sparse table when **all** of these are true:

Condition	Why
The array is static (no updates)	Sparse table cannot handle updates efficiently
The operation is idempotent or associative with overlap allowed	min, max, gcd, bitwise AND, bitwise OR
You need many queries (often 10^5 – 10^6)	$O(1)$ per query is the goal
You need fast online queries	No offline preprocessing of queries needed

Common Trigger Phrases from Problems

- "Given an array, answer Q queries of min/max/GCD in range [L, R]"
- "No updates between queries"
- "Static array range queries"
- "Find the minimum in all subarrays of length k"
- "Answer quickly — many queries"
- "Lowest common ancestor of two nodes in a tree" (LCA with Euler tour + sparse table)

4. When Not to Use It

Situation	Why Not	Better Alternative
Array values change between queries	Sparse table must be rebuilt from scratch $O(n \log n)$ per update	Segment Tree ($O(\log n)$ update)
Operation is sum, XOR, product (non-idempotent)	Overlapping intervals double-count	Segment Tree, Fenwick Tree

Situation	Why Not	Better Alternative
You need to answer range queries with updates	Sparse table is static	Segment Tree, Fenwick Tree
n is very small (≤ 1000) and Q is small	Precomputation overhead not worth it	Brute force per query ($O(n)$) is simpler
Memory is very tight	Sparse table uses $O(n \log n)$ memory	Square Root Decomposition ($O(n)$ memory)
Operation is not associative (e.g., median)	Cannot combine intervals easily	Mo's algorithm, persistent segment tree

Common Wrong Assumptions

- ✗ "Sparse table works for sum queries" — It does not. Overlap breaks sum.
- ✗ "I can update one element and keep using the table" — You cannot. Rebuild from scratch.
- ✗ "Sparse table is always faster than segment tree" — Only for queries. If updates are needed, segment tree wins.

5. Core Concepts

5.1 Powers of Two

The entire structure hinges on intervals whose lengths are powers of two: 1, 2, 4, 8, 16, ..., 2^k .

- Why it matters:** Any integer length can be expressed as a sum of powers of two (binary representation). For idempotent ops, the largest single power-of-two interval placed at both ends is enough.
- Example:** Length 11 = 8 + 2 + 1. But we don't need all three; we just use the largest one (8) twice.

5.2 The `st` Table

A 2D array where `st[k][i]` = answer for range `[i, i + 2k - 1]`.

- Dimensions: $(K+1) \times n$ where $K = \text{floor}(\log_2(n))$.
- `st[0][i] = arr[i]`
- `st[k][i] = op(st[k-1][i], st[k-1][i + 2(k-1)])`
- Why it matters:** This is the core data structure. Understanding how it's indexed and built is the whole algorithm.

5.3 Idempotence

An operation `op` is **idempotent** if `op(a, a) = a` for all `a`.

- Examples: `min(a, a) = a`, `max(a, a) = a`, `gcd(a, a) = a`, `a & a = a`, `a | a = a`.
- Non-examples: `sum(a, a) = 2a`, `xor(a, a) = 0`.

- **Why it matters:** Overlapping intervals are fine only if applying the operation to the same element twice does not change the result.

5.4 Range Reduction for Queries

For a query $[L, R]$:

1. $len = R - L + 1$
2. $k = \text{floor}(\log_2(len))$
3. $\text{Answer} = \text{op}(\text{st}[k][L], \text{st}[k][R - 2^k + 1])$

- **Why it matters:** This is the $O(1)$ query trick. The two intervals $[L, L + 2^k - 1]$ and $[R - 2^k + 1, R]$ cover the whole range and always overlap (or touch).

5.5 Log Table

Precompute $\log_2[i]$ for $i = 1..n$ to avoid calling $\log_2()$ at query time (which is slow and has floating-point issues).

- $\log_2[1] = 0$
- For $i \geq 2$: $\log_2[i] = \log_2[i/2] + 1$
- **Why it matters:** $O(1)$ query with no floating point, just integer arithmetic.

6. Step-by-Step Algorithm

Preprocessing

1. **Read** the input array `arr` of size n .
2. **Compute** $K = \text{floor}(\log_2(n))$.
3. **Allocate** $\text{st}[K+1][n]$ (or a vector of vectors).
4. **Initialize** $\text{st}[0][i] = \text{arr}[i]$ for all i .
5. **Precompute** $\log_2[1..n]$ table.
6. **Build** the table:
 - For $k = 1$ to K :
 - For $i = 0$ to $n - 2^k$:
 - $\text{st}[k][i] = \text{op}(\text{st}[k-1][i], \text{st}[k-1][i + 2^{(k-1)}])$

Query

1. Given L, R (0-indexed inclusive).
2. $length = R - L + 1$
3. $k = \log_2[length]$
4. $\text{return } \text{op}(\text{st}[k][L], \text{st}[k][R - (1 \ll k) + 1])$

7. Dry Run

Array: `arr = [5, 2, 4, 7, 1, 3, 6, 0]`, $n = 8$

Operation: Range Minimum Query (RMQ)

Precomputation

$K = \text{floor}(\log_2(8)) = 3$

st[0] (length = 1):

i	0	1	2	3	4	5	6	7
st[0]	5	2	4	7	1	3	6	0

st[1] (length = 2):

- $\text{st}[1][0] = \min(5, 2) = 2$
- $\text{st}[1][1] = \min(2, 4) = 2$
- $\text{st}[1][2] = \min(4, 7) = 4$
- $\text{st}[1][3] = \min(7, 1) = 1$
- $\text{st}[1][4] = \min(1, 3) = 1$
- $\text{st}[1][5] = \min(3, 6) = 3$
- $\text{st}[1][6] = \min(6, 0) = 0$

i	0	1	2	3	4	5	6
st[1]	2	2	4	1	1	3	0

st[2] (length = 4):

- $\text{st}[2][0] = \min(2, 4) = 2$
- $\text{st}[2][1] = \min(2, 1) = 1$
- $\text{st}[2][2] = \min(4, 1) = 1$
- $\text{st}[2][3] = \min(1, 3) = 1$
- $\text{st}[2][4] = \min(1, 0) = 0$

i	0	1	2	3	4
st[2]	2	1	1	1	0

st[3] (length = 8):

- $\text{st}[3][0] = \min(2, 0) = 0$

i	0
st[3]	0

Query: $\min(1, 6) \rightarrow$ indices $[1, 6]$, 0-indexed

- $\text{length} = 6 - 1 + 1 = 6$
- $k = \text{floor}(\log_2(6)) = 2$ (since $2^2 = 4$)
- $\text{st}[2][1] = 1$ (range $[1, 4]$)
- $\text{st}[2][6 - 4 + 1] = \text{st}[2][3] = 1$ (range $[3, 6]$)
- $\min(1, 1) = 1$

Result: 1  (actual min of [2, 4, 7, 1, 3, 6] is 1)

Query: min(0, 3) → indices [0, 3]

- `length = 4`
- `k = floor(log2(4)) = 2`
- `st[2][0] = 2` (range [0, 3])
- `st[2][3 - 4 + 1] = st[2][0] = 2` (same interval)
- `min(2, 2) = 2`

Result: 2  (actual min of [5, 2, 4, 7] is 2)

Query: min(4, 7)

- `length = 4`
- `k = 2`
- `st[2][4] = 0` (range [4, 7])
- `st[2][7 - 4 + 1] = st[2][4] = 0`
- `min(0, 0) = 0`

Result: 0 

8. C++ Implementation

Below is a reusable sparse table implementation for range minimum queries. To switch to max or GCD, change the `merge` function.

```
#include <bits/stdc++.h>
using namespace std;

class SparseTable {
private:
    int n;                // size of original array
    int K;                // floor(log2(n))
    vector<vector<int>>> st; // sparse table: st[k][i]
    vector<int> log;       // precomputed log2 values

    // Change this function for different operations
    static int merge(int a, int b) {
        return min(a, b); // For RMQ
        // return max(a, b); // For range max query
        // return gcd(a, b); // For GCD query
        // return a & b; // For bitwise AND
        // return a | b; // For bitwise OR
    }

public:
    SparseTable(const vector<int>& arr) {
        n = arr.size();
        if (n == 0) return;
        K = floor(log2(n));
        st.resize(K+1, vector<int>(n));
        log.resize(n);
        for (int i = 0; i < n; i++) log[i] = 0;
        for (int k = 1; k <= K; k++) {
            for (int i = 0; i + (1<<k) < n; i++) {
                st[k][i] = merge(st[k-1][i], st[k-1][i + (1<<(k-1))]);
            }
        }
    }
};
```

```

    // Precompute log2 values
    log.resize(n + 1);
    log[1] = 0;
    for (int i = 2; i <= n; i++) {
        log[i] = log[i / 2] + 1;
    }

    K = log[n];

    // Build sparse table
    st.assign(K + 1, vector<int>(n));
    for (int i = 0; i < n; i++) {
        st[0][i] = arr[i];
    }

    for (int k = 1; k <= K; k++) {
        int len = 1 << k;           // 2^k
        int half = 1 << (k - 1);    // 2^(k-1)
        for (int i = 0; i + len - 1 < n; i++) {
            st[k][i] = merge(st[k - 1][i], st[k - 1][i + half]);
        }
    }
}

// Query on range [L, R] inclusive (0-indexed)
int query(int L, int R) const {
    assert(L >= 0 && R < n && L <= R);
    int len = R - L + 1;
    int k = log[len];
    // Two overlapping power-of-two intervals
    return merge(st[k][L], st[k][R - (1 << k) + 1]);
}

// Direct access to the sparse table (useful for some advanced queries)
int get(int k, int i) const {
    return st[k][i];
}
};

// --- Example Usage ---
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int> arr = {5, 2, 4, 7, 1, 3, 6, 0};
    SparseTable st(arr);

    cout << "Minimum in [1, 6]: " << st.query(1, 6) << "\n"; // 1
    cout << "Minimum in [0, 3]: " << st.query(0, 3) << "\n"; // 2
    cout << "Minimum in [4, 7]: " << st.query(4, 7) << "\n"; // 0
    cout << "Minimum in [2, 2]: " << st.query(2, 2) << "\n"; // 4
}

```

```
    return 0;
}
```

For Range GCD Query

Just change `merge`:

```
static int merge(int a, int b) {
    return gcd(a, b);
}
```

For Range Max Query

```
static int merge(int a, int b) {
    return max(a, b);
}
```

9. Python Implementation

```
import math
from typing import List

class SparseTable:
    """Sparse Table for static range queries (min, max, gcd, etc.)"""

    def __init__(self, arr: List[int]):
        self.n = len(arr)
        if self.n == 0:
            return

        # Precompute log2 values
        self.log = [0] * (self.n + 1)
        for i in range(2, self.n + 1):
            self.log[i] = self.log[i // 2] + 1

        self.K = self.log[self.n]

        # Build sparse table
        # st[k][i] = answer for range [i, i + 2^k - 1]
        self.st = [[0] * self.n for _ in range(self.K + 1)]

        # Level 0: intervals of length 1
        for i in range(self.n):
            self.st[0][i] = arr[i]

        # Higher levels
```

```

    for k in range(1, self.K + 1):
        half = 1 << (k - 1) # 2^(k-1)
        # i + 2^k - 1 must be < n
        for i in range(self.n - (1 << k) + 1):
            # Merge two smaller intervals
            self.st[k][i] = self._merge(
                self.st[k - 1][i],
                self.st[k - 1][i + half]
            )

def _merge(self, a: int, b: int) -> int:
    """Override this for different operations."""
    return min(a, b) # RMQ

def query(self, L: int, R: int) -> int:
    """Query on range [L, R] inclusive (0-indexed)."""
    assert 0 <= L <= R < self.n
    length = R - L + 1
    k = self.log[length]
    # Two overlapping intervals cover the whole range
    return self._merge(
        self.st[k][L],
        self.st[k][R - (1 << k) + 1]
    )

# --- Example Usage ---
if __name__ == "__main__":
    arr = [5, 2, 4, 7, 1, 3, 6, 0]
    st = SparseTable(arr)

    print(f"Minimum in [1, 6]: {st.query(1, 6)}") # 1
    print(f"Minimum in [0, 3]: {st.query(0, 3)}") # 2
    print(f"Minimum in [4, 7]: {st.query(4, 7)}") # 0
    print(f"Minimum in [2, 2]: {st.query(2, 2)}") # 4

```

For Range GCD Query

```

import math

def _merge(self, a: int, b: int) -> int:
    return math.gcd(a, b)

```

For Range Max Query

```

def _merge(self, a: int, b: int) -> int:
    return max(a, b)

```

10. Code Explanation

C++ Breakdown

Class structure:

- `n, K`: Store array size and $\log_2(n)$. These define the table dimensions.
- `st`: 2D vector where `st[k][i]` stores the answer for interval $[i, i + 2^k - 1]$.
- `log`: Precomputed integer logs. `log[len] = floor(log2(len))`. Avoids calling `log2()` at query time.

Constructor:

1. Precompute `log[i]` for all `i` from 1 to `n`:
 - `log[1] = 0`
 - For `i >= 2`: `log[i] = log[i/2] + 1` (integer division). This works because `floor(log2(i))` increases by 1 when `i` doubles.
2. Set `K = log[n]` — the maximum power level needed.
3. Resize `st` to `(K+1)` rows × `n` columns.
4. Fill `st[0][i] = arr[i]` (base intervals of length 1).
5. Build higher levels:
 - For each `k`, an interval of length 2^k is split into two intervals of length 2^{k-1} .
 - `st[k][i] = merge(st[k-1][i], st[k-1][i + half])`
 - Loop only up to `i + 2k - 1 < n` to stay in bounds.

Query method:

1. `len = R - L + 1`
2. `k = log[len]` → largest power of two $\leq$ len
3. First interval: `st[k][L]` covers $[L, L + 2^k - 1]$
4. Second interval: `st[k][R - 2k + 1]` covers $[R - 2^k + 1, R]$
5. Return `merge(first, second)`
6. These two intervals always cover $[L, R]$ and overlap (or touch exactly when len is a power of two).

merge function:

- This is the pluggable part. Change it to switch between min, max, gcd, AND, OR.
- In a real codebase, you'd use a template parameter or `std::function`. For CP/placements, a simple `static` method is clean enough.

Python Breakdown

- Same logic, translated to Python.
- Uses list comprehensions and explicit loops.
- `_merge` method is designed to be overridden in subclass or monkey-patched.
- Precomputation uses integer arithmetic for logs.

11. Complexity Analysis

Operation	Time	Space
Preprocessing (Build)	$O(n \log n)$	$O(n \log n)$
Single Query	$O(1)$	—
Update	Not supported	—

Detailed Breakdown

Preprocessing:

- Outer loop: $K + 1 \approx \log_2(n)$ iterations
- Inner loop (level k): $n - 2^k + 1 \approx n$ iterations
- Total work: $n * (K+1) \approx n \log_2 n$

Query:

- $\log$ table lookup: $O(1)$
- Two array accesses: $O(1)$
- One merge operation: $O(1)$

Space:

- `st` table: $(K+1) \times n \approx n \log_2 n$ integers
- $\log$ table: $n + 1$ integers

Best case: Same as worst case — the table is always fully built.

Worst case: $O(n \log n)$ for build, which is unavoidable for a dense static table.

12. Common Patterns

Pattern 1: Range Min/Max Query (Static)

- How to identify:** "Given an array, answer Q queries of minimum/maximum in range $[L, R]$. No updates."
- Approach:** Build RMQ sparse table. $O(1)$ per query.
- Example problems:** Range Minimum Query (CSES), Static RMQ (Codeforces)

Pattern 2: Range GCD Query (Static)

- How to identify:** "Find GCD of all elements in range $[L, R]$. Queries are many, array is static."
- Approach:** Sparse table with `gcd` as merge function.
- Example problem:** Range GCD (Codeforces, SPOJ)

Pattern 3: LCA using Sparse Table (Euler Tour)

- How to identify:** "Find lowest common ancestor of two nodes in a tree. Multiple queries."
- Approach:**
 - Perform Euler tour of the tree (DFS, record node each time you visit it).
 - Record the depth of each node at each position in the Euler tour.

3. Record the first occurrence of each node in the tour.
4. LCA of u and v = the node with minimum depth in the Euler tour between `first[u]` and `first[v]`.
5. Use sparse table for range minimum query on depths.

- **Example problem:** LCA (CSES, SPOJ, Codeforces)

Pattern 4: Range AND/OR (Static)

- **How to identify:** "Find bitwise AND/OR of all elements in range $[L, R]$. Many queries."
- **Approach:** Sparse table with `&` or `|` as merge.
- **Why it works:** AND and OR are idempotent.
- **Example problem:** Range Bitwise AND (LeetCode)

Pattern 5: Number of Distinct Elements in Static Array Subarrays

- **How to identify:** "Count distinct elements in range $[L, R]$ — array is static."
- **Approach:** This is NOT directly doable with sparse table (distinct is not idempotent). Use Mo's algorithm or a persistent segment tree instead.
- **Why mentioned:** Common trap — students try sparse table for distinct count and fail.

13. Common Mistakes

✗ Using Sparse Table for Non-Idempotent Operations

```
// WRONG: Sum is not idempotent!
return st[k][L] + st[k][R - (1 << k) + 1]; // Overlap double-counts
```

Fix: Use segment tree or Fenwick tree for sum queries.

✗ Off-by-One in Query Indices

```
// WRONG: R - (1 << k) should have +1
st[k][L], st[k][R - (1 << k)] // Missing +1

// CORRECT:
st[k][L], st[k][R - (1 << k) + 1]
```

✗ Wrong Log Precomputation

```
// WRONG:
log[2] = 1; // Should be 1? Let's check:
// floor(log2(2)) = 1. Yes, correct.
// floor(log2(3)) = 1. Using log[3] = log[1] + 1 = 0 + 1 = 1. Correct.

// But this is wrong:
```

```
log[i] = log[i - 1] + (i & (i - 1) == 0); // Too complex, error-prone

// Standard correct way:
log[1] = 0;
for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;
```

✗ Out-of-Bounds During Build

```
// WRONG: doesn't check boundary
for (int i = 0; i < n; i++) {
    st[k][i] = merge(st[k-1][i], st[k-1][i + half]);
}

// CORRECT:
for (int i = 0; i + (1 << k) - 1 < n; i++) {
    st[k][i] = merge(st[k-1][i], st[k-1][i + half]);
}
```

✗ Using 1-Indexed vs 0-Indexed Confusion

Be consistent. The implementation above assumes 0-indexed. If the problem gives 1-indexed queries, subtract 1 from L and R.

✗ Not Handling Empty Array

Always check $n == 0$ before building the table.

✗ Integer Overflow in Shifts

```
1 << k // Safe if  $k < 31$  (int). Use  $1LL << k$  for larger.
```

For arrays up to 10^5 , $k \leq 16$, so int is safe.

✗ Memory Explosion

st as a 2D vector of size $(\log n + 1) \times n$ can be large. For $n = 10^5$, that's about $17 \times 10^5 = 1.7$ million ints (~6.8 MB). For $n = 10^6$, ~20 million ints (~80 MB). Still okay for most platforms, but be mindful on memory-constrained judges.

14. Edge Cases

Edge Case	Example	Expected Behavior
Single element	<code>arr = [42]</code> , <code>query(0, 0)</code>	Return 42
All equal elements	<code>arr = [5, 5, 5, 5]</code>	Query returns 5

Edge Case	Example	Expected Behavior
Sorted ascending	<code>arr = [1, 2, 3, 4, 5]</code>	Min = L, Max = R, works fine
Sorted descending	<code>arr = [5, 4, 3, 2, 1]</code>	Same, works fine
Full array query	<code>query(0, n-1)</code>	Works — $\log[n]$ gives correct k
Length = power of two	<code>query(0, 3)</code> when $n=4$	The two intervals coincide: <code>st[2][0]</code> and <code>st[2][0]</code>
$L == R$	Single element query	<code>len = 1, k = 0, st[0][L]</code> is returned
Large values	<code>arr[i]</code> up to 10^9	Use <code>int</code> (fits in 32-bit signed) or <code>long long</code>
Negative values	<code>arr = [-5, -2, -8, -1]</code>	Min = -8, Max = -1 — works fine
$n = 0$ (empty array)	No elements	Constructor should handle gracefully
$n = 1$	Single element, $K = 0$	Only <code>st[0]</code> exists. <code>query(0,0)</code> works with $\log[1] = 0$

15. Variations

15.1 2D Sparse Table (Range Query on Matrix)

- **What changes:** Build sparse table on rows and columns. Queries take $O(1)$ for idempotent ops on a static matrix.
- **When used:** "Given a static matrix, answer Q queries of min/max in submatrix."
- **Placements/CP:** Rare but appears in hard problems. Not essential for most placements.

15.2 Disjoint Sparse Table (Non-Idempotent + Static)

- **What changes:** Instead of overlapping, the table stores answers for intervals that break at binary boundaries. Queries are $O(1)$ and support non-idempotent ops (like sum, XOR).
- **When used:** "Static array, many queries, operation is sum/XOR, no updates."
- **Placements/CP:** Advanced. Good to know exist, but segment tree is simpler and sufficient.

15.3 Sparse Table on Trees (LCA)

- **What changes:** Use Euler tour to flatten the tree. Then build a sparse table on the depth array. Query = node with minimum depth in the Euler range.
- **When used:** LCA queries on a static tree.
- **Placements/CP: Very important.** LCA is a classic and common topic.

15.4 Sparse Table on Strings (Longest Common Prefix / LCP Array)

- **What changes:** Build RMQ sparse table on the LCP array of a suffix array. Allows $O(1)$ longest common prefix queries between any two suffixes.
- **When used:** String processing, suffix array problems.
- **Placements/CP:** Advanced, but useful in string-heavy rounds.

16. Related Algorithms/Data Structures

Data Structure	Static Queries	Updates	Idempotent Required	Query Time	Build Time
Sparse Table	✓ Yes	✗ No	✓ Yes	$O(1)$	$O(n \log n)$
Segment Tree	✓ Yes	✓ Yes	✗ No	$O(\log n)$	$O(n)$
Fenwick Tree	✓ Sum only	✓ Yes	✗ No	$O(\log n)$	$O(n)$
Mo's Algorithm	✓ Yes	✗ No	✗ No	$O(\sqrt{n})$ per query (offline)	—
Square Root Decomposition	✓ Yes	✓ Yes	✗ No	$O(\sqrt{n})$	$O(n)$
Disjoint Sparse Table	✓ Yes	✗ No	✗ No	$O(1)$	$O(n \log n)$

How to Choose

Your Need	Choose
Static array, idempotent queries (min/max/gcd), many queries	Sparse Table
Array with updates, any associative operation	Segment Tree
Static array with sum queries (no updates)	Prefix Sum ($O(1)$ query, $O(n)$ build, no log factor)
Array with updates, sum queries	Fenwick Tree or Segment Tree
Offline queries, any operation (updates optional)	Mo's Algorithm
LCA on a static tree	Sparse Table on Euler tour or Binary Lifting
LCA with updates on tree (dynamic)	HLD + Segment Tree

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Range Minimum Query	CSES	Static RMQ — direct application	★ Easy
Static Range Maximum Queries (actually RMQ, but similar)	CSES	Static Max queries	★ Easy

Medium

Problem	Platform	Pattern	Difficulty
LCA — Lowest Common Ancestor	CSES	Euler tour + sparse table for RMQ on depths	☆☆ Medium
Range GCD	CodeChef	GCD sparse table (note: updates make it segment tree, but static version is sparse table)	☆☆ Medium
XOR and Queries	Codeforces	Bitwise AND/OR with sparse table	☆☆ Medium

Hard

Problem	Platform	Pattern	Difficulty
Xenia and Tree	Codeforces	LCA + Centroid Decomposition (sparse table for LCA)	☆☆☆ Hard
Sereja and Brackets	Codeforces	Segment tree on bracket sequences (not sparse table — but a good contrast problem)	☆☆☆ Hard
Powerful array	Codeforces	Mo's algorithm (good contrast — distinct count is NOT sparse-table-friendly)	☆☆☆ Hard

18. Interview Explanation

"A sparse table is a data structure for static range queries. It precomputes answers for intervals whose lengths are powers of two — so for an array of size n , we have a table of size $\log n$ by n . Each cell `st[k][i]` stores the answer for the interval starting at i of length 2^k .

To build it: `st[0][i]` is just `arr[i]`, and for $k \geq 1$, `st[k][i] = op(st[k-1][i], st[k-1][i + 2^(k-1)])`. This is $O(n \log n)$.

To query a range $[L, R]$: let $k = \text{floor}(\log_2(\text{len}))$. The answer is `op(st[k][L], st[k][R - 2^k + 1])`. These two intervals cover the whole range and may overlap — but for operations like min, max, or gcd, overlapping doesn't change the result because they're idempotent. Query is $O(1)$.

The trade-off is that updates are not supported — you'd rebuild from scratch. So it's only suitable for static arrays. For dynamic arrays or sum-like operations, you'd use a segment tree instead."

19. Revision Notes

Key Idea

- Precompute answers for intervals of length 1, 2, 4, 8, ..., 2^k
- Query: cover $[L, R]$ with two overlapping power-of-two intervals
- Only for **idempotent** operations (min, max, gcd, AND, OR)

Formula

```
st[k][i] = op(st[k-1][i], st[k-1][i + 2^(k-1)])
```

```
query(L, R):
```

```
    k = floor(log2(R - L + 1))
```

```
    return op(st[k][L], st[k][R - 2^k + 1])
```

Precompute Log Table

```
log[1] = 0
```

```
for i = 2..n: log[i] = log[i/2] + 1
```

Complexity

- **Build:** $O(n \log n)$ time, $O(n \log n)$ space
- **Query:** $O(1)$ time

Common Traps

- ❌ Don't use for sum/XOR — not idempotent, overlap breaks it
- ❌ Don't use with updates — static only
- ❌ Off-by-one: $R - (1 \ll k) + 1$, not $R - (1 \ll k)$
- ❌ Forgot to handle single-element ($k=0$) queries
- ❌ Forgot bounds during build: $i + (1 \ll k) - 1 < n$

LCA with Sparse Table

1. Euler tour (DFS, record node at each visit)
2. Depth array (depth of node at each Euler position)
3. `first[node]` = first index where node appears in Euler array
4. `LCA(u, v)` = node with min depth in Euler range `[first[u], first[v]]`
5. Use RMQ sparse table on the depth array

20. Final Cheat Sheet

SPARSE TABLE — CHEAT SHEET

WHEN TO USE:

- Static array (no updates)
- Many range queries (min, max, gcd, AND, OR)
- Need $O(1)$ per query

MAIN OPERATIONS:

- `build(arr) →  $O(n \log n)$`
- `query(L, R) →  $O(1)$`
- `update → NOT SUPPORTED`

SPACE: $O(n \log n)$

KEY CODE:

```
// Build
for i: st[0][i] = arr[i]
for k: for i: st[k][i] = merge(st[k-1][i], st[k-1][i+half])

// Query
k = log[R - L + 1]
ans = merge(st[k][L], st[k][R - (1<<k) + 1])
```

IMPORTANT EDGE CASES:

- Single element: `len=1, k=0`, same interval twice
- Power-of-two length: both intervals are identical
- Empty array: guard with `n==0` check
- Negative values: min/max handle correctly

TRAPS:

- X Sum/XOR → double-counts on overlap
- X Updates → rebuild entire table
- X Off-by-one in `R - 2k + 1`
- X 1-indexed vs 0-indexed confusion

LCA TRICK:

Euler tour → depth array → RMQ sparse table
 LCA = min-depth node between `first[u]` and `first[v]`

Static Range Minimum Query (RMQ)

1. Overview

Static Range Minimum Query (RMQ) is the classic application of a sparse table. Given a fixed array `arr[0..n-1]`, answer queries of the form: "What is the smallest element in `arr[L..R]`?" — with no updates to the array between queries.

The answer is returned as the **value** of the minimum element (or sometimes its **index**, depending on the problem).

2. Intuition

The minimum operation has two crucial properties:

1. **Associative:** $\min(a, \min(b, c)) = \min(\min(a, b), c)$ — order doesn't matter.
2. **Idempotent:** $\min(a, a) = a$ — repeating an element doesn't change the result.

Because of idempotence, we can let the two precomputed power-of-two intervals **overlap**. The minimum of the two intervals is guaranteed to be the minimum of the whole range, even if some elements are counted twice.

Analogy: If you want the shortest person in a row of people, and you ask two overlapping groups, taking the shorter of the two group-minimums always gives the correct global minimum — even if some people are in both groups.

3. When to Use It

- Static array, many queries (10^5+)
- Need $O(1)$ query time
- Need to find minimum value (or its index) in any subarray
- As a building block for LCA, longest common prefix, etc.

Trigger Phrases

- "Find the minimum in range $[L, R]$ "
- "No updates"
- "Answer Q queries"
- "Lowest value in the segment"

4. When Not to Use It

- Array updates are required → use a **segment tree**
- You need the sum or another non-idempotent operation → use prefix sums or segment tree
- Only a few queries on a small array → brute force $O(n)$ per query is simpler and might be fast enough
- The problem asks for the **second minimum** or **k-th minimum** — sparse table cannot answer these directly

5. Core Concepts

5.1 The **st** Table for Min

$st[k][i]$ = minimum value in $arr[i..i + 2^k - 1]$.

- Base: $st[0][i] = arr[i]$
- Recurrence: $st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}])$

5.2 Query Reduction

$query(L, R)$:

- $k = \text{floor}(\log_2(R - L + 1))$
- Result = $\min(st[k][L], st[k][R - 2^k + 1])$

5.3 Index of Minimum (Index RMQ)

Sometimes you need the **index** of the minimum, not the value. Build the sparse table on the values, but store the index instead. When merging, compare values and pick the index with the smaller value.

```
st_idx[k][i] = index of min in arr[i..i+2^k-1]
merge: compare arr[idx1] and arr[idx2], pick index with smaller value
```

6. Step-by-Step Algorithm

Preprocessing

1. Let `arr[0..n-1]` be the input.
2. `K = floor(log2(n))`
3. For `i` in `0..n-1`: `st[0][i] = arr[i]`
4. For `k = 1..K`:
 - `step = 2^(k-1)`
 - For `i` in `0..n-2^k`:
 - `st[k][i] = min(st[k-1][i], st[k-1][i + step])`

Query

1. Input: `L, R` (0-indexed inclusive)
2. `len = R - L + 1`
3. `k = log2[len]`
4. Return `min(st[k][L], st[k][R - (1<<k) + 1])`

7. Dry Run

Already covered in the main sparse table dry run (Section 7). The array `[5, 2, 4, 7, 1, 3, 6, 0]` with RMQ is the exact example shown there.

8. C++ Implementation (RMQ)

See the main `SparseTable` class in Section 8 — it uses `min` as the merge function.

For **Index RMQ**:

```
#include <bits/stdc++.h>
using namespace std;

class IndexRMQ {
private:
    int n;
    int K;
```

```

vector<vector<int>> st; // stores indices, not values
vector<int> log;
const vector<int>& arr; // reference to original array

int mergeIdx(int i, int j) const {
    return (arr[i] <= arr[j]) ? i : j; // pick index of smaller value
}

public:
    IndexRMQ(const vector<int>& a) : arr(a) {
        n = a.size();
        if (n == 0) return;

        log.resize(n + 1);
        log[1] = 0;
        for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;

        K = log[n];
        st.assign(K + 1, vector<int>(n));
        for (int i = 0; i < n; i++) st[0][i] = i;

        for (int k = 1; k <= K; k++) {
            int half = 1 << (k - 1);
            for (int i = 0; i + (1 << k) - 1 < n; i++) {
                st[k][i] = mergeIdx(st[k - 1][i], st[k - 1][i + half]);
            }
        }
    }

    // Returns the index of the minimum in [L, R]
    int queryIdx(int L, int R) const {
        int len = R - L + 1;
        int k = log[len];
        return mergeIdx(st[k][L], st[k][R - (1 << k) + 1]);
    }

    // Returns the value of the minimum in [L, R]
    int queryVal(int L, int R) const {
        return arr[queryIdx(L, R)];
    }
};

```

9. Python Implementation (RMQ)

Already covered in Section 9. The `SparseTable` class uses `min` as default.

For Index RMQ in Python:

```

class IndexRMQ:
    def __init__(self, arr: List[int]):

```

```

self.arr = arr
self.n = len(arr)
if self.n == 0:
    return
self.log = [0] * (self.n + 1)
for i in range(2, self.n + 1):
    self.log[i] = self.log[i // 2] + 1
self.K = self.log[self.n]
self.st = [[0] * self.n for _ in range(self.K + 1)]
for i in range(self.n):
    self.st[0][i] = i
for k in range(1, self.K + 1):
    half = 1 << (k - 1)
    for i in range(self.n - (1 << k) + 1):
        left = self.st[k - 1][i]
        right = self.st[k - 1][i + half]
        self.st[k][i] = left if arr[left] <= arr[right] else right

def query_idx(self, L: int, R: int) -> int:
    k = self.log[R - L + 1]
    left = self.st[k][L]
    right = self.st[k][R - (1 << k) + 1]
    return left if self.arr[left] <= self.arr[right] else right

def query_val(self, L: int, R: int) -> int:
    return self.arr[self.query_idx(L, R)]

```

10. Code Explanation

The RMQ sparse table follows the exact same code structure as the generic sparse table.

Key difference in Index RMQ: The table stores indices, and the merge compares the values at those indices. This is useful when you need to know **where** the minimum occurs, not just its value.

Edge case in merge: When values are equal, either index works. The code above picks the leftmost by using `<=` (preferring smaller index). This is helpful in LCA (picking the shallower node when depths are equal).

11. Complexity Analysis

Operation	Time	Space
Build	$O(n \log n)$	$O(n \log n)$
Query (value or index)	$O(1)$	—

Same as generic sparse table.

12. Common Patterns

1. **Direct RMQ** — value of minimum in range
2. **Index RMQ** — position of minimum in range (used in LCA, LCP)
3. **Range minimum of multiple arrays** — independent sparse tables on each
4. **Minimum in sliding window** — can also use deque, but sparse table works for static

13. Common Mistakes

- Using RMQ sparse table for range sum (overlap double-counts)
- Forgetting that two intervals overlap and the operation must be idempotent
- Not handling $L == R$ correctly (len=1, k=0, same interval appears twice)
- Building with wrong bounds: $i + (1 \ll k) - 1 < n$

14. Edge Cases

Case	Example	Notes
Single element	[5], query(0,0)	Returns 5 (or index 0)
All equal	[3,3,3,3]	Any index is correct
Decreasing	[9,8,7,6]	Min at the end
Increasing	[1,2,3,4]	Min at the start
Negative values	[-5,-2,-8]	Min = -8

15. Variations

15.1 2D RMQ

For a static matrix, precompute min for powers of two in both dimensions. Query $O(1)$ for any submatrix.

15.2 RMQ with Cartesian Tree

Can be used to build a Cartesian tree from RMQ queries in $O(n)$. Not common in placements.

16. Related Algorithms

Algorithm	When to Use
Sparse Table (RMQ)	Static array, $O(1)$ queries
Segment Tree	Dynamic array, $O(\log n)$ queries + updates
Square Root Decomposition	Simpler, $O(\sqrt{n})$ queries, supports updates
Fenwick Tree	Only for sum, not min

17. Practice Problems

Easy

- **Range Minimum Query** — CSES — Direct $O(1)$ query after building sparse table

Medium

- **LCA using RMQ** — CSES — Euler tour + RMQ sparse table on depths
- **Longest Common Prefix of Suffixes** — Requires RMQ on LCP array

Hard

- **Xenia and Tree** — Codeforces 342E — LCA + centroid decomposition with RMQ
- **Tree Queries** — CSES — Requires LCA (RMQ) + difference arrays

18. Interview Explanation

"For static range minimum queries, I'd use a sparse table. It precomputes the minimum for every interval whose length is a power of two — that's about $\log n$ levels, each with n intervals, so $O(n \log n)$ time and space. To answer a query on $[L, R]$, I take the largest power of two that fits in that range, and take the minimum of two overlapping intervals of that size starting from L and ending at R respectively. This works because $\min$ is idempotent — overlapping doesn't matter. The query is $O(1)$. If the array had updates, I'd switch to a segment tree with $O(\log n)$ per operation."

19. Revision Notes

- RMQ = **min** operation on sparse table
- Index RMQ stores indices, compares by value
- Same build/query logic as generic sparse table
- $O(1)$ query, $O(n \log n)$ build
- Cannot handle updates

20. RMQ Cheat Sheet

RMQ SPARSE TABLE:

```
st[k][i] = min in arr[i..i+2^k-1]
query(L,R) = min(st[k][L], st[k][R-2^k+1])
              where k = floor(log2(R-L+1))
```

INDEX RMQ:

```
st[k][i] = index of min in arr[i..i+2^k-1]
merge: pick index with smaller arr value
(prefer leftmost on tie)
```

Static Range Maximum Query

1. Overview

Static Range Maximum Query is the mirror of RMQ. Given a fixed array, answer queries for the maximum value (or its index) in any range $[L, R]$. Everything about RMQ applies directly — just replace `min` with `max`.

2. Intuition

Same as RMQ, but looking for the largest element. `max` is also idempotent: $\text{max}(a, a) = a$. Overlapping intervals are safe.

Analogy: Finding the tallest person in a row. Asking two overlapping groups and taking the taller of the two maxima gives the correct global maximum.

3. When to Use It

- Find maximum in a static subarray
 - Many queries, no updates
 - Combined with RMQ to get both min and max of a range (e.g., "find the difference between min and max in range")
-

4. When Not to Use It

- Same restrictions as RMQ: no updates, idempotent only
-

5. Core Concepts

Exactly the same as RMQ (Section 5), but:

- $\text{st}[k][i] = \text{max}(\text{st}[k-1][i], \text{st}[k-1][i + 2^{(k-1)}])$
 - $\text{query}(L, R) = \text{max}(\text{st}[k][L], \text{st}[k][R - 2^k + 1])$
-

6. Step-by-Step Algorithm

Same as RMQ (Section 6), replacing `min` with `max`.

7. Dry Run

Array: `[5, 2, 4, 7, 1, 3, 6, 0]`

st[0]: `[5, 2, 4, 7, 1, 3, 6, 0]`

st[1] (length 2):

- $\text{max}(5, 2)=5, \text{max}(2, 4)=4, \text{max}(4, 7)=7, \text{max}(7, 1)=7, \text{max}(1, 3)=3, \text{max}(3, 6)=6, \text{max}(6, 0)=6$
- `[5, 4, 7, 7, 3, 6, 6]`

st[2] (length 4):

- $\max(5, 7)=7, \max(4, 7)=7, \max(7, 3)=7, \max(7, 6)=7, \max(3, 6)=6$
- $[7, 7, 7, 7, 6]$

st[3] (length 8):

- $\max(7, 6)=7$
- $[7]$

Query $\max(1, 6)$: $\text{len}=6, k=2, \max(\text{st}[2][1]=7, \text{st}[2][3]=7) = 7$ ✓

Query $\max(0, 3)$: $\text{len}=4, k=2, \max(\text{st}[2][0]=7, \text{st}[2][0]=7) = 7$ ✓

8. C++ Implementation

Simply change the `merge` function:

```
static int merge(int a, int b) {
    return max(a, b);
}
```

Use the same `SparseTable` class from Section 8.

9. Python Implementation

Change `_merge`:

```
def _merge(self, a: int, b: int) -> int:
    return max(a, b)
```

Use the same `SparseTable` class from Section 9.

10. Complexity Analysis

Operation	Time	Space
Build	$O(n \log n)$	$O(n \log n)$
Query	$O(1)$	—

11. Cheat Sheet

MAX SPARSE TABLE:

```
st[k][i] = max in arr[i..i+2^k-1]
query(L,R) = max(st[k][L], st[k][R-2^k+1])
Same as RMQ, just min → max
```

GCD Range Query

1. Overview

Given a static array, answer queries of the form: "What is the GCD of all elements in `arr[L..R]`?"

GCD is idempotent: $\text{gcd}(a, a) = a$. So the sparse table approach works directly.

2. Intuition

GCD of a range can be computed by dividing the range into two overlapping power-of-two intervals and taking GCD of the two results. Because $\text{gcd}(a, a) = a$ and GCD is associative, overlapping doesn't cause any problems.

Analogy: If you want the GCD of numbers 1 through 10, and you compute GCD of (1..8) = g_1 and GCD of (3..10) = g_2 , then $\text{gcd}(g_1, g_2) = \text{GCD of } (1..10)$. The overlap [3..8] just contributes its GCD twice, which doesn't change anything.

3. When to Use It

- Static array, many GCD range queries
- "Find GCD of elements in range [L, R]"
- "Array is fixed, no updates"

4. When Not to Use It

- Array has updates → use **segment tree** with GCD as merge
- Need **LCM** of range → LCM is not idempotent ($\text{lcm}(a, a) = a$ actually is idempotent, but LCM of overlapping intervals is not straightforward because $\text{lcm}(\text{lcm}(A), \text{lcm}(B)) \neq \text{lcm}(A \cup B)$ when A and B overlap — wait, actually lcm is also idempotent. But the issue is that $\text{lcm}(g_1, g_2)$ for overlapping ranges might give wrong result because LCM is not associative over GCD-like decomposition... Let me be precise: **lcm** IS idempotent and associative, so sparse table would work mathematically. But in practice, LCM can overflow very quickly and is rarely queried in static settings. Use segment tree for safety.)
- Need GCD of a range with element-wise updates → segment tree

Actually, let me correct the above: **lcm** is idempotent ($\text{lcm}(a, a) = a$) and associative, so sparse table would work for LCM too. The main practical concern is overflow. But for placements, GCD is the one that appears

frequently.

5. Core Concepts

5.1 GCD Properties

- $\text{gcd}(a, b) = \text{gcd}(b, a)$ (commutative)
- $\text{gcd}(a, \text{gcd}(b, c)) = \text{gcd}(\text{gcd}(a, b), c)$ (associative)
- $\text{gcd}(a, a) = a$ (idempotent)
- $\text{gcd}(0, a) = a$

5.2 Sparse Table for GCD

Same structure:

- $\text{st}[k][i] = \text{gcd}(\text{arr}[i..i + 2^k - 1])$
 - $\text{query}(L, R) = \text{gcd}(\text{st}[k][L], \text{st}[k][R - 2^k + 1])$
-

6. Step-by-Step Algorithm

Same as RMQ (Section 6), replacing `min` with `gcd`.

Use `std::gcd` (C++17) or `__gcd` (older) or `math.gcd` (Python).

7. Dry Run

Array: [12, 18, 6, 24, 36, 8, 16, 10]

st[0]: [12, 18, 6, 24, 36, 8, 16, 10]

st[1] (length 2):

- $\text{gcd}(12, 18)=6, \text{gcd}(18, 6)=6, \text{gcd}(6, 24)=6, \text{gcd}(24, 36)=12, \text{gcd}(36, 8)=4, \text{gcd}(8, 16)=8, \text{gcd}(16, 10)=2$
- [6, 6, 6, 12, 4, 8, 2]

st[2] (length 4):

- $\text{gcd}(6, 6)=6, \text{gcd}(6, 12)=6, \text{gcd}(6, 4)=2, \text{gcd}(12, 8)=4, \text{gcd}(4, 2)=2$
- [6, 6, 2, 4, 2]

st[3] (length 8):

- $\text{gcd}(6, 2)=2$
- [2]

Query gcd(0, 3): len=4, k=2, $\text{gcd}(\text{st}[2][0]=6, \text{st}[2][0]=6) = 6$ ✓ (GCD of [12,18,6,24] = 6)

Query gcd(1, 5): len=5, k=2, $\text{gcd}(\text{st}[2][1]=6, \text{st}[2][2]=2) = 2$ ✓ (GCD of [18,6,24,36,8] = 2)

Query gcd(4, 7): $\text{len}=4, k=2, \text{gcd}(\text{st}[2][4]=2, \text{st}[2][4]=2) = 2$ ✓ (GCD of [36,8,16,10] = 2)

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class GCDSparseTable {
private:
    int n, K;
    vector<vector<int>> st;
    vector<int> log;

public:
    GCDSparseTable(const vector<int>& arr) {
        n = arr.size();
        if (n == 0) return;

        log.resize(n + 1);
        log[1] = 0;
        for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;

        K = log[n];
        st.assign(K + 1, vector<int>(n));

        for (int i = 0; i < n; i++) st[0][i] = arr[i];

        for (int k = 1; k <= K; k++) {
            int half = 1 << (k - 1);
            for (int i = 0; i + (1 << k) - 1 < n; i++) {
                st[k][i] = gcd(st[k - 1][i], st[k - 1][i + half]);
            }
        }
    }

    int query(int L, int R) const {
        int len = R - L + 1;
        int k = log[len];
        return gcd(st[k][L], st[k][R - (1 << k) + 1]);
    }
};

// --- Example ---
int main() {
    vector<int> arr = {12, 18, 6, 24, 36, 8, 16, 10};
    GCDSparseTable st(arr);
    cout << st.query(0, 3) << "\n"; // 6
    cout << st.query(1, 5) << "\n"; // 2
    cout << st.query(4, 7) << "\n"; // 2
    return 0;
}

```

9. Python Implementation

```

import math
from typing import List

class GCDSparseTable:
    def __init__(self, arr: List[int]):
        self.n = len(arr)
        if self.n == 0:
            return
        self.log = [0] * (self.n + 1)
        for i in range(2, self.n + 1):
            self.log[i] = self.log[i // 2] + 1
        self.K = self.log[self.n]
        self.st = [[0] * self.n for _ in range(self.K + 1)]
        for i in range(self.n):
            self.st[0][i] = arr[i]
        for k in range(1, self.K + 1):
            half = 1 << (k - 1)
            for i in range(self.n - (1 << k) + 1):
                self.st[k][i] = math.gcd(
                    self.st[k - 1][i],
                    self.st[k - 1][i + half]
                )

    def query(self, L: int, R: int) -> int:
        k = self.log[R - L + 1]
        return math.gcd(
            self.st[k][L],
            self.st[k][R - (1 << k) + 1]
        )

```

10. Complexity Analysis

Operation	Time	Space
Build	$O(n \log n)$	$O(n \log n)$
Query	$O(1)$ (gcd of two ints)	—

Note: gcd of two integers is $O(\log \min(a,b))$ in the worst case, but in practice $O(1)$ for fixed-width integers.

11. Cheat Sheet

GCD SPARSE TABLE:

```

st[k][i] = gcd(arr[i..i+2^k-1])
query(L,R) = gcd(st[k][L], st[k][R-2^k+1])
Same structure as RMQ, just min → gcd
Edge case: gcd(0, a) = a (handled automatically)

```

Idempotent Operations

1. Overview

An **idempotent operation** is one where applying the operation to the same value multiple times gives the same result as applying it once. Formally: $op(a, a) = a$ for all a .

This property is the **foundation** of why sparse tables work. Without idempotence, overlapping intervals would double-count elements and produce wrong answers.

2. Intuition

Simple Explanation

Imagine you're asked to find the coldest temperature in a week. If you look at Monday-Friday and Wednesday-Sunday separately, and then take the colder of the two results, you'll get the right answer — even though Wednesday-Friday appears in both ranges. The coldest temperature stays the coldest no matter how many times you include it.

But if you were asked for the **total rainfall** in the week, and you added up Monday-Friday + Wednesday-Sunday, you'd double-count Wednesday-Friday's rain. That's wrong. That's the difference between idempotent (min) and non-idempotent (sum).

Analogy

- **Idempotent:** "What's the minimum/maximum?" — Taking the min of two overlapping subranges gives the correct global min.
- **Non-idempotent:** "What's the sum?" — Adding two overlapping subranges double-counts the overlap.

3. Which Operations Are Idempotent?

Operation	Idempotent?	Reason
$\min(a, b)$	✓	$\min(a, a) = a$
$\max(a, b)$	✓	$\max(a, a) = a$
$\gcd(a, b)$	✓	$\gcd(a, a) = a$
$\text{lcm}(a, b)$	✓	$\text{lcm}(a, a) = a$
$a \ \& \ b$ (bitwise AND)	✓	$a \ \& \ a = a$
$a \ \ b$ (bitwise OR)	✓	$a \ \ a = a$
$a + b$ (sum)	✗	$a + a = 2a \neq a$
$a \ ^ \wedge \ b$ (XOR)	✗	$a \ ^ \wedge \ a = 0 \neq a$

Operation	Idempotent?	Reason
$a * b$ (product)	✗	$a * a = a^2 \neq a$ (unless $a=0$ or 1)
<code>count distinct</code>	✗	Not even associative

4. When to Use Idempotent Operations with Sparse Table

Use a sparse table when:

- The merge operation is **idempotent** AND **associative**
- The array is static
- You need $O(1)$ queries

5. When Not to Use

- Operation is not idempotent → use segment tree (for associative ops) or Fenwick tree (for sum/xor)
- Operation is not associative (e.g., median) → sparse table doesn't work regardless

6. Core Concepts

6.1 Associativity vs Idempotence

Property	Definition	Example
Associativity	$op(a, op(b, c)) = op(op(a, b), c)$	All operations listed above are associative
Idempotence	$op(a, a) = a$	min, max, gcd, AND, OR

Both are needed for the overlapping-interval trick. Associativity ensures we can combine intervals. Idempotence ensures overlap doesn't break correctness.

6.2 The Overlap Argument

For query $[L, R]$ with interval length len :

- Let $k = \text{floor}(\log_2(len))$
- Interval $A = [L, L + 2^k - 1]$
- Interval $B = [R - 2^k + 1, R]$

$$\text{Length of overlap} = 2^k + 2^k - len = 2^{k+1} - len$$

Since $2^k \leq len < 2^{k+1}$, the overlap is always positive (or zero when len is a power of two).

For idempotent operations, the overlap doesn't matter. For non-idempotent, it does.

7. Step-by-Step: Checking Idempotence

To check if an operation `op` is idempotent:

1. Pick any value a
2. Compute $op(a, a)$
3. If the result equals a for all a , it's idempotent

8. C++ Implementation

```
// Template-based sparse table that works with any idempotent operation
#include <bits/stdc++.h>
using namespace std;

template<typename T, T (*op)(T, T)>
class IdempotentSparseTable {
private:
    int n, K;
    vector<vector<T>> st;
    vector<int> log;

public:
    IdempotentSparseTable(const vector<T>& arr) {
        n = arr.size();
        if (n == 0) return;

        log.resize(n + 1);
        log[1] = 0;
        for (int i = 2; i <= n; i++) log[i] = log[i / 2] + 1;

        K = log[n];
        st.assign(K + 1, vector<T>(n));

        for (int i = 0; i < n; i++) st[0][i] = arr[i];

        for (int k = 1; k <= K; k++) {
            int half = 1 << (k - 1);
            for (int i = 0; i + (1 << k) - 1 < n; i++) {
                st[k][i] = op(st[k - 1][i], st[k - 1][i + half]);
            }
        }

        T query(int L, int R) const {
            int len = R - L + 1;
            int k = log[len];
            return op(st[k][L], st[k][R - (1 << k) + 1]);
        }
    };

    // Operation functions
    int minOp(int a, int b) { return min(a, b); }
    int maxOp(int a, int b) { return max(a, b); }
    int gcdOp(int a, int b) { return gcd(a, b); }
    int andOp(int a, int b) { return a & b; }
```

```
int orOp(int a, int b) { return a | b; }

// Usage:
// IdempotentSparseTable<int, minOp> rmq(arr);
// IdempotentSparseTable<int, maxOp> rmaxq(arr);
// IdempotentSparseTable<int, gcdOp> rgcdq(arr);
```

9. Python Implementation

```
from typing import Callable, List, TypeVar

T = TypeVar('T')

class IdempotentSparseTable:
    def __init__(self, arr: List[T], op: Callable[[T, T], T]):
        self.arr = arr
        self.op = op
        self.n = len(arr)
        if self.n == 0:
            return
        self.log = [0] * (self.n + 1)
        for i in range(2, self.n + 1):
            self.log[i] = self.log[i // 2] + 1
        self.K = self.log[self.n]
        self.st = [[0] * self.n for _ in range(self.K + 1)]
        for i in range(self.n):
            self.st[0][i] = arr[i]
        for k in range(1, self.K + 1):
            half = 1 << (k - 1)
            for i in range(self.n - (1 << k) + 1):
                self.st[k][i] = op(
                    self.st[k - 1][i],
                    self.st[k - 1][i + half]
                )

    def query(self, L: int, R: int) -> T:
        k = self.log[R - L + 1]
        return self.op(
            self.st[k][L],
            self.st[k][R - (1 << k) + 1]
        )

# Usage:
# rmq = IdempotentSparseTable(arr, min)
# rmaxq = IdempotentSparseTable(arr, max)
# rgcd = IdempotentSparseTable(arr, math.gcd)
```

10. Complexity Analysis

Same as RMQ: $O(n \log n)$ build, $O(1)$ query.

11. Cheat Sheet

IDEMPOTENT OPERATIONS FOR SPARSE TABLE:

- ✓ min, max, gcd, AND, OR
- ✗ sum, XOR, product, count distinct

Key property: $op(a, a) = a$

Overlap is safe because elements in overlap contribute the same value twice.

LCA with Sparse Table

1. Overview

The **Lowest Common Ancestor (LCA)** of two nodes in a tree is the deepest node that is an ancestor of both. LCA with a sparse table is a two-step process:

1. **Flatten the tree** using an Euler tour (DFS).
2. **Build an RMQ sparse table** on the depths of the Euler tour.

Query: $LCA(u, v)$ = the node with minimum depth in the Euler tour range between the first occurrence of u and the first occurrence of v .

This approach answers LCA queries in **$O(1)$** after **$O(n \log n)$** preprocessing.

2. Intuition

Simple Explanation

When you do a DFS on a tree and record each node every time you visit it (the Euler tour), you get an array where each node appears multiple times. Between any two occurrences of two nodes in this array, the **minimum depth node** is their LCA.

Why? Because the Euler tour traces the entire DFS path. The range between $first(u)$ and $first(v)$ includes the path from u up to the LCA and back down to v . The shallowest node along that subpath is the LCA.

Analogy

Imagine walking through a tree, jotting down every node you visit. If you mark when you first see each node, then between the first sightings of two nodes, the highest (shallowest) node you pass through is their common ancestor — the point from which both branches diverge.

3. When to Use It

- Need to answer many LCA queries on a static tree
- Tree size up to 10^5 – 10^6 , queries up to 10^5 – 10^6
- Need $O(1)$ per query after preprocessing
- Building block for problems involving:
 - Distance between two nodes ($\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$)
 - Path queries (with difference arrays or HLD)
 - Tree DP with LCA

4. When Not to Use It

- Tree is being modified (add/remove nodes) → use **Link-Cut Tree** or **dynamic LCA**
- Only need LCA once or twice → binary lifting ($O(\log n)$ per query, simpler to implement)
- n is small → binary lifting is simpler
- Memory is tight → binary lifting uses $O(n \log n)$ too, but Euler + sparse table uses $\sim 2x$ more memory (Euler array is $2n-1$ elements)

Comparison: Binary Lifting vs Sparse Table LCA

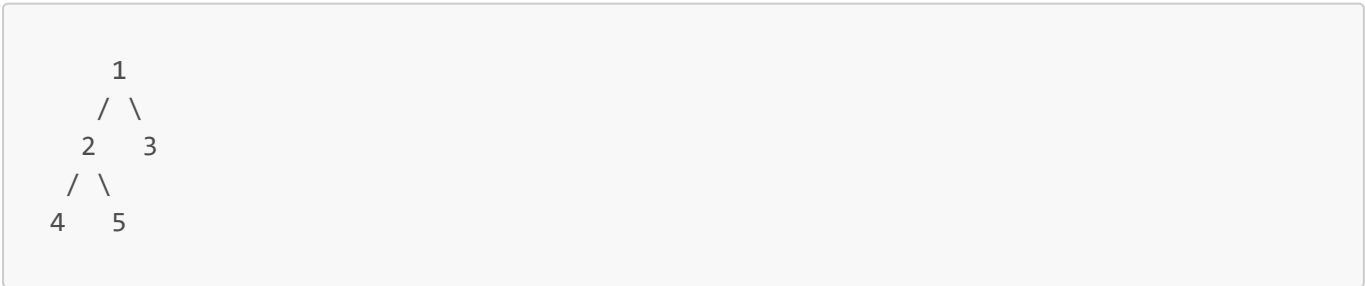
Aspect	Binary Lifting	Euler Tour + Sparse Table
Preprocessing	$O(n \log n)$	$O(n \log n)$
Query	$O(\log n)$	$O(1)$
Memory	$O(n \log n)$	$O(n \log n)$
Simplicity	Simpler to code	Requires Euler tour + sparse table
When to use	Sufficient for most placements	When $O(1)$ per query matters

5. Core Concepts

5.1 Euler Tour (DFS Order)

A DFS traversal that records each node every time it is **visited** (including on entry and return).

For a tree:



Euler tour (entry only): $[1, 2, 4, 2, 5, 2, 1, 3, 1]$ Wait — let me be precise. The standard Euler tour for LCA records each node every time it's visited:

```

Start at 1: visit 1
Go to 2: visit 2
  Go to 4: visit 4
  Back to 2: visit 2
  Go to 5: visit 5
  Back to 2: visit 2
Back to 1: visit 1
Go to 3: visit 3
Back to 1: visit 1

```

Euler array: `[1, 2, 4, 2, 5, 2, 1, 3, 1]`, length = $2n - 1 = 7$... wait, that's 9 elements for $n=5$. Let me recount.

Actually: `1-2-4-2-5-2-1-3-1` — that's 9 elements. For $n=5$, $2n-1 = 9$. ✓

5.2 Depth Array

For each position in the Euler array, store the depth of the node at that position.

```

Euler:  [1, 2, 4, 2, 5, 2, 1, 3, 1]
Depth:  [0, 1, 2, 1, 2, 1, 0, 1, 0]

```

5.3 First Occurrence Array

`first[node]` = the first index in the Euler array where `node` appears.

```

first[1] = 0
first[2] = 1
first[3] = 7
first[4] = 2
first[5] = 4

```

5.4 RMQ on Depths

Build a sparse table on the depth array. `st[k][i]` = index of the minimum depth in the range `[i, i + 2k - 1]` (Index RMQ, because we need the node, not just the depth).

5.5 LCA Query

`lca(u, v)`:

1. `l = min(first[u], first[v]), r = max(first[u], first[v])`
2. `idx = queryIndexRMQ(l, r)` — index of minimum depth in Euler tour range
3. `return euler[idx]` — the node at that position is the LCA

6. Step-by-Step Algorithm

Preprocessing

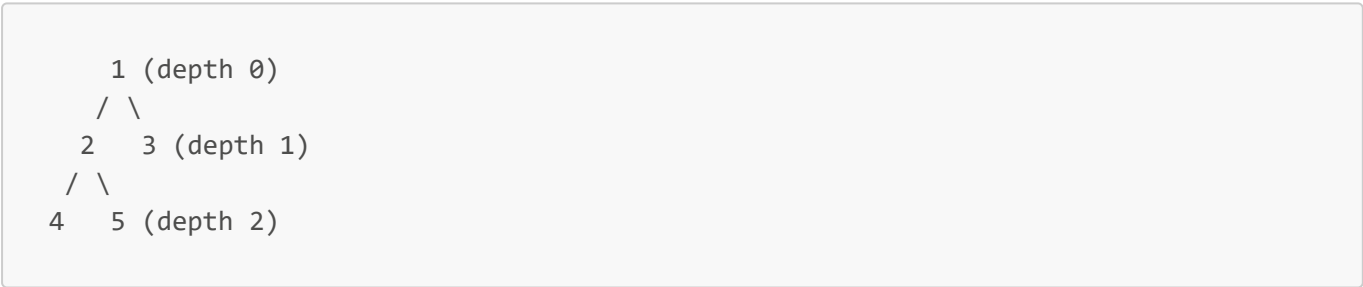
1. Run DFS from root (usually node 1 or 0):
 - Push current node to `euler` array
 - Record `depth[euler.size() - 1] = nodeDepth`
 - If `first[node]` is not set, set it
 - For each child, DFS into child, then push current node again (backtrack)
2. Build Index RMQ sparse table on the `depth` array.

Query

1. `l = first[u], r = first[v]`
2. If `l > r`, swap them.
3. `idx = rmq.queryIdx(l, r)` — index of minimum depth
4. Return `euler[idx]`

7. Dry Run

Tree:



DFS (Euler Tour)

Starting from 1, visiting children left-to-right:

Step	Current	Action	Euler	Depth	first
1	1	Enter 1	[1]	[0]	first[1]=0
2	2	Enter 2	[1,2]	[0,1]	first[2]=1
3	4	Enter 4	[1,2,4]	[0,1,2]	first[4]=2
4	4	Exit 4 → back to 2	[1,2,4,2]	[0,1,2,1]	—
5	5	Enter 5	[1,2,4,2,5]	[0,1,2,1,2]	first[5]=4
6	5	Exit 5 → back to 2	[1,2,4,2,5,2]	[0,1,2,1,2,1]	—
7	2	Exit 2 → back to 1	[1,2,4,2,5,2,1]	[0,1,2,1,2,1,0]	—
8	3	Enter 3	[1,2,4,2,5,2,1,3]	[0,1,2,1,2,1,0,1]	first[3]=7
9	3	Exit 3 → back to 1	[1,2,4,2,5,2,1,3,1]	[0,1,2,1,2,1,0,1,0]	—

Final arrays:

```
euler = [1, 2, 4, 2, 5, 2, 1, 3, 1]
depth = [0, 1, 2, 1, 2, 1, 0, 1, 0]
first = {1:0, 2:1, 3:7, 4:2, 5:4}
```

Sparse Table on Depths (Index RMQ)

Build $st[k][i]$ = index of minimum depth in $depth[i..i+2^k-1]$.

st[0]: [0, 1, 2, 3, 4, 5, 6, 7, 8] (depth at each index: [0, 1, 2, 1, 2, 1, 0, 1, 0])

st[1] (length 2): compare depth[idx] at pairs

- $i=0$: $\min(\text{depth}[0]=0, \text{depth}[1]=1) \rightarrow \text{idx } 0$
- $i=1$: $\min(\text{depth}[1]=1, \text{depth}[2]=2) \rightarrow \text{idx } 1$
- $i=2$: $\min(\text{depth}[2]=2, \text{depth}[3]=1) \rightarrow \text{idx } 3$
- $i=3$: $\min(\text{depth}[3]=1, \text{depth}[4]=2) \rightarrow \text{idx } 3$
- $i=4$: $\min(\text{depth}[4]=2, \text{depth}[5]=1) \rightarrow \text{idx } 5$
- $i=5$: $\min(\text{depth}[5]=1, \text{depth}[6]=0) \rightarrow \text{idx } 6$
- $i=6$: $\min(\text{depth}[6]=0, \text{depth}[7]=1) \rightarrow \text{idx } 6$
- $i=7$: $\min(\text{depth}[7]=1, \text{depth}[8]=0) \rightarrow \text{idx } 8$

st[1] = [0, 1, 3, 3, 5, 6, 6, 8]

st[2] (length 4): compare st[1] values

- $i=0$: min depth at $st[1][0]=0$ (depth 0) vs $st[1][2]=3$ (depth 1) $\rightarrow \text{idx } 0$
- $i=1$: min depth at $st[1][1]=1$ (depth 1) vs $st[1][3]=3$ (depth 1) $\rightarrow \text{idx } 1$ (leftmost)
- $i=2$: min depth at $st[1][2]=3$ (depth 1) vs $st[1][4]=5$ (depth 2) $\rightarrow \text{idx } 3$
- $i=3$: min depth at $st[1][3]=3$ (depth 1) vs $st[1][5]=6$ (depth 0) $\rightarrow \text{idx } 6$
- $i=4$: min depth at $st[1][4]=5$ (depth 2) vs $st[1][6]=6$ (depth 0) $\rightarrow \text{idx } 6$
- $i=5$: min depth at $st[1][5]=6$ (depth 0) vs $st[1][7]=8$ (depth 0) $\rightarrow \text{idx } 6$ (leftmost)

st[2] = [0, 1, 3, 6, 6, 6]

st[3] (length 8):

- $i=0$: min depth at $st[2][0]=0$ (depth 0) vs $st[2][4]=6$ (depth 0) $\rightarrow \text{idx } 0$

st[3] = [0]

Query: LCA(4, 5)

1. $l = \text{first}[4] = 2, r = \text{first}[5] = 4$
2. $\text{len} = 3, k = \text{floor}(\log_2(3)) = 1$
3. $\text{idx1} = \text{st}[1][2] = 3, \text{idx2} = \text{st}[1][4 - 2 + 1] = \text{st}[1][3] = 3$
4. Both give index 3. $\text{euler}[3] = 2$
5. LCA(4, 5) = 2  (Node 2 is the parent of both 4 and 5)

Query: LCA(4, 3)

1. $l = \text{first}[4] = 2, r = \text{first}[3] = 7$
2. $\text{len} = 6, k = \text{floor}(\log_2(6)) = 2$
3. $\text{idx1} = \text{st}[2][2] = 3, \text{idx2} = \text{st}[2][7 - 4 + 1] = \text{st}[2][4] = 6$
4. Compare depth at 3 (=1) vs depth at 6 (=0). Min is at index 6 $\rightarrow \text{euler}[6] = 1$
5. LCA(4, 3) = 1 

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class LCA {
private:
    int n;
    vector<vector<int>> adj;
    vector<int> euler;      // Euler tour array (nodes)
    vector<int> depth;      // depth at each Euler position
    vector<int> first;      // first occurrence of each node in Euler tour

    // Index RMQ sparse table on depth array
    vector<vector<int>> st;
    vector<int> log;

    void dfs(int u, int p, int d) {
        first[u] = euler.size();
        euler.push_back(u);
        depth.push_back(d);

        for (int v : adj[u]) {
            if (v == p) continue;
            dfs(v, u, d + 1);
            euler.push_back(u);
            depth.push_back(d);
        }
    }

    void buildRMQ() {
        int m = euler.size();
        log.resize(m + 1);
        log[1] = 0;
        for (int i = 2; i <= m; i++) log[i] = log[i / 2] + 1;

        int K = log[m];
        st.assign(K + 1, vector<int>(m));

        // st[k][i] = index of minimum depth in range [i, i+2^k-1]
        for (int i = 0; i < m; i++) st[0][i] = i;

        for (int k = 1; k <= K; k++) {
```

```

        int half = 1 << (k - 1);
        for (int i = 0; i + (1 << k) - 1 < m; i++) {
            int left = st[k - 1][i];
            int right = st[k - 1][i + half];
            // Pick index with smaller depth; prefer leftmost on tie
            st[k][i] = (depth[left] <= depth[right]) ? left : right;
        }
    }
}

public:
    LCA(int n, const vector<vector<int>>& adj, int root = 1)
        : n(n), adj(adj) {
        first.assign(n + 1, -1);
        dfs(root, -1, 0);
        buildRMQ();
    }

    int lca(int u, int v) const {
        int l = first[u];
        int r = first[v];
        if (l > r) swap(l, r);

        int len = r - l + 1;
        int k = log[len];

        int leftIdx = st[k][l];
        int rightIdx = st[k][r - (1 << k) + 1];

        int minIdx = (depth[leftIdx] <= depth[rightIdx]) ? leftIdx : rightIdx;
        return euler[minIdx];
    }

    int distance(int u, int v) const {
        int w = lca(u, v);
        // Need actual node depths: depth[first[u]] gives depth of u
        int du = depth[first[u]];
        int dv = depth[first[v]];
        int dw = depth[first[w]];
        return du + dv - 2 * dw;
    }

    // Get depth of a node (using first occurrence)
    int getDepth(int u) const {
        return depth[first[u]];
    }
};

// --- Example Usage ---
int main() {
    int n = 5;
    vector<vector<int>> adj(n + 1);
    adj[1] = {2, 3};
    adj[2] = {1, 4, 5};

```

```

adj[3] = {1};
adj[4] = {2};
adj[5] = {2};

LCA lca(n, adj, 1);

cout << "LCA(4, 5): " << lca.lca(4, 5) << "\n"; // 2
cout << "LCA(4, 3): " << lca.lca(4, 3) << "\n"; // 1
cout << "LCA(2, 3): " << lca.lca(2, 3) << "\n"; // 1

cout << "Distance(4, 5): " << lca.distance(4, 5) << "\n"; // 2
cout << "Distance(4, 3): " << lca.distance(4, 3) << "\n"; // 3

return 0;
}

```

9. Python Implementation

```

from typing import List
import sys
sys.setrecursionlimit(10**6)

class LCA:
    def __init__(self, n: int, adj: List[List[int]], root: int = 1):
        self.n = n
        self.adj = adj
        self.euler = []
        self.depth = []
        self.first = [-1] * (n + 1)

        self._dfs(root, -1, 0)
        self._build_rmq()

    def _dfs(self, u: int, p: int, d: int):
        self.first[u] = len(self.euler)
        self.euler.append(u)
        self.depth.append(d)

        for v in self.adj[u]:
            if v == p:
                continue
            self._dfs(v, u, d + 1)
            self.euler.append(u)
            self.depth.append(d)

    def _build_rmq(self):
        m = len(self.euler)
        self.log = [0] * (m + 1)
        for i in range(2, m + 1):
            self.log[i] = self.log[i // 2] + 1

```

```

K = self.log[m]
# st[k][i] = index of min depth in range [i, i+2^k-1]
self.st = [[0] * m for _ in range(K + 1)]
for i in range(m):
    self.st[0][i] = i

for k in range(1, K + 1):
    half = 1 << (k - 1)
    for i in range(m - (1 << k) + 1):
        left = self.st[k - 1][i]
        right = self.st[k - 1][i + half]
        self.st[k][i] = left if self.depth[left] <= self.depth[right] else
right

def lca(self, u: int, v: int) -> int:
    l = self.first[u]
    r = self.first[v]
    if l > r:
        l, r = r, l

    length = r - l + 1
    k = self.log[length]

    left_idx = self.st[k][l]
    right_idx = self.st[k][r - (1 << k) + 1]

    min_idx = left_idx if self.depth[left_idx] <= self.depth[right_idx] else
right_idx
    return self.euler[min_idx]

def distance(self, u: int, v: int) -> int:
    w = self.lca(u, v)
    du = self.depth[self.first[u]]
    dv = self.depth[self.first[v]]
    dw = self.depth[self.first[w]]
    return du + dv - 2 * dw

# --- Example ---
if __name__ == "__main__":
    n = 5
    adj = [[] for _ in range(n + 1)]
    adj[1] = [2, 3]
    adj[2] = [1, 4, 5]
    adj[3] = [1]
    adj[4] = [2]
    adj[5] = [2]

    lca = LCA(n, adj, 1)

    print(f"LCA(4, 5): {lca.lca(4, 5)}") # 2
    print(f"LCA(4, 3): {lca.lca(4, 3)}") # 1

```

```
print(f"Distance(4, 5): {lca.distance(4, 5)}") # 2
print(f"Distance(4, 3): {lca.distance(4, 3)}") # 3
```

10. Code Explanation

DFS Phase

- **euler**: Records nodes during DFS traversal. Each node appears multiple times — once when first entered, and again each time we return from a child.
- **depth**: Parallel array to **euler**, storing the depth of the node at each position.
- **first**: Maps each node to its first index in **euler**.

The DFS visits children left-to-right (order doesn't matter for correctness).

RMQ Build Phase

- Build an **Index RMQ** sparse table on the **depth** array.
- **st[k][i]** stores the **index** (position in **euler**) where the minimum depth occurs in range $[i, i + 2^k - 1]$.
- When depths are equal, we prefer the leftmost index (using $\leq$). This doesn't affect LCA correctness.

LCA Query

- $l = \text{first}[u], r = \text{first}[v]$ (ensure $l \leq r$).
- Query the range $[l, r]$ in the Euler tour for the node with minimum depth.
- That node is the LCA.

Distance Query

- $\text{dist}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}(u, v)]$
- Uses $\text{depth}[\text{first}[\text{node}]]$ to get the actual depth of a node.

11. Complexity Analysis

Operation	Time	Space
DFS (Euler tour)	$O(n)$	$O(n)$
Build RMQ sparse table	$O(n \log n)$	$O(n \log n)$
LCA Query	$O(1)$	—
Distance Query	$O(1)$	—

Total preprocessing: $O(n \log n)$ **Total memory:** $O(n \log n)$ for sparse table + $O(n)$ for Euler/depth/first arrays

12. Common Patterns

Pattern 1: Distance Between Nodes

```
dist(u, v) = depth[u] + depth[v] - 2 * depth[lca(u, v)]
```

Pattern 2: Path Queries with Difference Arrays

- Sum of values on path from u to v:
 - $\text{sum}[u] + \text{sum}[v] - 2 * \text{sum}[\text{lca}] + \text{val}[\text{lca}]$ (if values on nodes)
 - Using prefix sums from root + LCA

Pattern 3: K-th Ancestor

- Can be done with binary lifting (not sparse table LCA)
- But sparse table LCA gives the LCA, then use binary lifting for the k-th ancestor

Pattern 4: Subtree Queries

- Euler tour (entry + exit times) → flatten subtree into a contiguous range
- Then use segment tree or Fenwick tree on the flattened array

13. Common Mistakes

✗ Using Full Euler Tour (Including Returns) as a Simple Array

The Euler tour includes backtracking. `first[node]` maps to the first occurrence. Using the wrong occurrence (e.g., the last) can give wrong LCA.

✗ Forgetting to Swap l and r

Always ensure $l \leq r$ before querying the RMQ.

✗ Off-by-One in Sparse Table Query

```
// WRONG:
st[k][R - (1 << k)] // Missing +1

// CORRECT:
st[k][R - (1 << k) + 1]
```

✗ Recursion Depth in DFS

For large trees ($n = 10^5$), recursion depth can exceed stack limit. Use iterative DFS or increase recursion limit.

In Python, use `sys.setrecursionlimit(10**6)`. In C++, the default stack is usually fine for 10^5 , but for 10^6 , use iterative stack.

✗ 0-Indexed vs 1-Indexed Confusion

The tree nodes are typically 1-indexed. **first** array size should be $n + 1$.

14. Edge Cases

Case	Notes
$u == v$	LCA is the node itself. Range is single element.
u is ancestor of v	$LCA = u$. The minimum in range will be u 's depth.
v is ancestor of u	Same as above.
Root with one child (chain)	Tree is a linked list. DFS depth goes 0,1,2,... Works fine.
Single node ($n=1$)	$LCA(1,1) = 1$. Euler array = [1], depth = [0].
Very deep tree ($n=10^5$)	Recursion may overflow. Use iterative DFS.

15. Variations

15.1 LCA with Binary Lifting

- **What changes:** No Euler tour. Precompute $up[k][v] = 2^k$ -th ancestor of v . Query by lifting both nodes to same depth, then binary search.
- **When used:** Simpler to implement, $O(\log n)$ per query. Good enough for most placements.
- **CP importance:** Very common. Both this and sparse table LCA are important.

15.2 LCA with Segment Tree

- **What changes:** Instead of sparse table, use a segment tree on the Euler depth array.
- **When used:** When you need more flexibility or are already using segment tree for other queries.
- **Complexity:** $O(\log n)$ per query.

15.3 LCA with Heavy-Light Decomposition (HLD)

- **What changes:** Decompose tree into heavy paths. LCA by climbing paths.
- **When used:** When you already need HLD for path queries (sum, max, etc.).
- **Complexity:** $O(\log n)$ per query.

15.4 Dynamic LCA

- **What changes:** Support adding/removing leaves or re-rooting.
- **When used:** Rare in placements, appears in some CP problems.
- **Approach:** Link-Cut Tree or Euler Tour Tree.

16. Related Algorithms

Algorithm	Query Time	Build Time	Supports Updates?	When to Use
Sparse Table LCA	$O(1)$	$O(n \log n)$	No	Many queries, static tree
Binary Lifting	$O(\log n)$	$O(n \log n)$	No	Simpler, moderate queries
Segment Tree LCA	$O(\log n)$	$O(n)$	No	Already using segtree
HLD LCA	$O(\log n)$	$O(n)$	No	Need path queries
Tarjan's LCA	$O(\alpha(n))$ amortized	$O(n)$	No	Offline queries

17. Practice Problems

Easy

Problem	Platform	Pattern	Difficulty
Lowest Common Ancestor	CSES	Direct LCA queries	★ Easy
Distance Queries	CSES	Distance = $\text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}]$	★ Easy

Medium

Problem	Platform	Pattern	Difficulty
Company Queries II	CSES	LCA in a company hierarchy	★★ Medium
Xenia and Tree	Codeforces	LCA + centroid decomposition	★★ Medium
Tree and Queries	Codeforces	LCA + Mo's algorithm on tree	★★ Medium

Hard

Problem	Platform	Pattern	Difficulty
Tree Queries	Codeforces	Virtual trees + LCA	★★★ Hard
Cave Exploration	Codeforces	2D sparse table + binary search	★★★ Hard

18. Interview Explanation

"LCA can be solved using a sparse table with $O(1)$ queries. The key insight is to flatten the tree into an Euler tour array — a DFS that records each node every time we visit it. This gives us an array of length $2n-1$. We also record the depth at each position and the first occurrence of each node.

To find LCA of u and v , we query the range between $\text{first}[u]$ and $\text{first}[v]$ on the depth array using a sparse table, looking for the index with minimum depth. That index's node is the LCA.

The sparse table is built in $O(n \log n)$ and answers queries in $O(1)$. The space is $O(n \log n)$. For an interview, I'd note that binary lifting is simpler to implement and uses $O(\log n)$ per query, which is usually sufficient."

19. Revision Notes

Key Idea

- Euler tour $\rightarrow$ depth array $\rightarrow$ RMQ sparse table
- LCA = node with min depth in Euler range between the two nodes

Euler Tour

```
DFS from root, record node at every visit (entry + backtracks)
euler = [1, 2, 4, 2, 5, 2, 1, 3, 1] (for the example tree)
depth = [0, 1, 2, 1, 2, 1, 0, 1, 0]
first[node] = first index of node in euler
```

Query

```
l = min(first[u], first[v])
r = max(first[u], first[v])
idx = RMQ(l, r) // min depth
return euler[idx]
```

Complexity

- Build: $O(n \log n)$ time, $O(n \log n)$ space
- Query: $O(1)$

Common Traps

- ✗ Forgetting `swap(l, r)` if `first[u] > first[v]`
- ✗ Index RMQ (store index of min depth), not value RMQ (store min depth)
- ✗ Recursion limit for deep trees (use iterative DFS or increase stack)
- ✗ First array of size $n+1$ (1-indexed nodes)

20. LCA Cheat Sheet

LCA WITH SPARSE TABLE:

PREPROCESS:

```
dfs(u, p, d):
    first[u] = euler.size()
```

```
euler.push(u), depth.push(d)
for v in adj[u]:
    if v != p:
        dfs(v, u, d+1)
        euler.push(u), depth.push(d)
```

Build Index-RMQ sparse table on depth[]

```
QUERY LCA(u, v):
    l = first[u], r = first[v]
    if l > r: swap(l, r)
    k = log2(r-l+1)
    i1 = st[k][l], i2 = st[k][r-(1<<k)+1]
    idx = i1 if depth[i1] <= depth[i2] else i2
    return euler[idx]

DISTANCE(u, v):
    return depth[u] + depth[v] - 2*depth[lca(u,v)]

(depth[node] = depth[first[node]])
```